KeyForge: Agentic Secret Management via FHE-Protected Credentials, HMAC Key Derivation, and Decentralized Proxy Validation

KeyForge Research Group
Decentralized Infrastructure Division
Technical Specification v1.0
2026

***Abstract*—The API credential management industry operates on a fundamentally broken trust model. Vendors issue static bearer tokens with no cryptographic enforcement of access rules. Rate limits, expiry, and usage constraints live in vendor databases enforced by policy, not mathematics. A single server compromise exposes every issued credential simultaneously, and AI agents that require plaintext secrets expose those secrets through context windows, chat logs, and training-data pipelines.**

This paper introduces KeyForge, the Agentic Secret Management (ASM) system: a decentralized API access marketplace that replaces vendor trust with cryptographic guarantees. Access tokens are derived via HMAC-SHA256 from a master key K_m protected by Fully Homomorphic Encryption (FHE) on-chain, validated offline by a Decentralized Physical Infrastructure Network (DePIN) of proxy nodes, and bound cryptographically to the buyer's wallet identity. Three core contributions are presented: (1) on-chain FHE storage of K_m using `euint256` with access-control-list (ACL) gating on Fhenix L2, (2) the Blind Courier Protocol for cryptographically blind credential handling by AI agents, and (3) a DePIN validation layer that eliminates single points of failure in request forwarding.

The system is post-quantum secure for the confidentiality of K_m via lattice-based TFHE, while the broader Ethereum infrastructure is not. Three open research questions are identified and honestly characterized: FHE-friendly MAC primitive selection, symmetric wrapping performance, and homomorphic rate-limit subdivision. The PoC implementation is fully buildable with current tooling; the full vendor-offline architecture awaits resolution of the MAC primitive problem.

***Index Terms*—Fully Homomorphic Encryption, API access control, DePIN, HMAC, AI agents, blockchain, credential management, Fhenix, CoFHE**

I. Introduction

API keys are the load-bearing credentials of the modern software stack. Every call to OpenAI, Stripe, Google Maps, or any programmatic service is authenticated by a static bearer token: a long random string that confers full access to whoever holds it. This model has three fundamental problems at scale.

First, **credentials are durable by default**. A leaked key is valid until manually revoked. GitHub's secret scanning service reports over 100,000 API keys exposed in public repositories annually [1]. The only remediation is revocation and reissuance, with no forensic trail of what was accessed in the interim.

Second, **rules live in the wrong place**. Rate limits, expiry windows, and usage constraints exist only in vendor databases. They are enforced by policy, not mathematics. A vendor server compromise or a dishonest vendor can silently violate every constraint without cryptographic detection.

Third, **AI agents cannot safely handle credentials**. Autonomous agents require plaintext secrets to call external APIs. Those secrets pass through model context windows, memory systems, and potentially training data pipelines. There is no cryptographic mechanism today that allows an agent to *use* a credential it *cannot read*.

KeyForge addresses all three through a composition of three well-understood primitives: FHE for on-chain credential confidentiality, HMAC-SHA256 for offline token derivation and validation, and a permissionless DePIN network for decentralized request forwarding. The result is an API access layer where credentials are enforced by math, master keys are unreadable by any party except their owner, and no single infrastructure component can compromise the system.

The remainder of this paper is structured as follows. Section II surveys related work. Section III describes the system architecture. Section IV details the cryptographic stack. Section V presents the Blind Courier Protocol. Section VI analyzes DePIN economics. Section VII provides a security analysis. Section VIII discusses the AI agent use case. Section IX characterizes open research problems. Section X outlines the implementation roadmap. Section XII concludes.

II. Related Work

A. Traditional API Gateways

Existing API management platforms such as Kong [2] and AWS API Gateway [3] provide rate limiting, authentication, and routing as centralized services. These systems improve on raw static keys but retain fundamental trust assumptions: all enforcement logic executes on vendor-controlled infrastructure, and credentials are stored in plaintext or reversibly encrypted. A single gateway compromise exposes all managed secrets. KeyForge replaces centralized enforcement with on-chain cryptographic rules and distributes validation across a permissionless node network.

B. Threshold Cryptography and Secret Management

Lit Protocol [4] provides decentralized threshold key management using multi-party computation (MPC). Users can

store encrypted secrets and define conditions under which they are decrypted. KeyForge's Threshold Services Network (TSN) component is directly analogous in function, differing in its integration with FHE-computed tokens and economic staking via EigenLayer. Shamir Secret Sharing [5] provides the theoretical foundation for threshold schemes; KeyForge adopts an $N/2$-of-N threshold model for the TSN.

C. FHE in Production Systems

Zama's TFHE library [6] and the Fhenix L2 [7] represent the current state of the art in production-grade FHE for blockchain applications. The CoFHE coprocessor architecture [8] offloads intensive FHE computation from the main execution thread, enabling practical latencies for simple operations (addition, comparison) while complex operations such as homomorphic MAC computation remain an open problem. BFV [9] complements TFHE for polynomial arithmetic workloads. KeyForge uses TFHE exclusively for master key storage and ACL operations.

D. DePIN Economic Models

Decentralized Physical Infrastructure Networks have demonstrated viable economic models across several domains. Helium [16] decentralizes wireless network infrastructure; Filecoin [17] decentralizes storage; Render Network [18] decentralizes GPU compute. Each demonstrates the core DePIN primitive: nodes stake collateral, provide a measurable service, earn fees proportional to service delivered, and face slashing for provable misbehavior. KeyForge applies this model to API request forwarding and validation.

E. Agentic Payment Protocols

The HTTP 402 Payment Required mechanism and its modern implementation x402 [15] enable per-request micropayments for API access. The protocol requires multiple round-trips: agent discovers payment requirement, initiates payment, receives receipt, includes receipt in retry, and vendor verifies payment before serving the response. For high-frequency agentic workloads (thousands of requests per hour), this overhead is prohibitive. KeyForge replaces per-request payment with session token issuance, bundling payment and access proof into a single token delivery.

III. System Architecture

A. The Glue and Coprocessor Model

KeyForge follows the "Glue and Coprocessor" architecture. The Ethereum-compatible Layer 2 (Fhenix L2) handles business logic and state management — the "glue" — while the CoFHE Coprocessor handles intensive encrypted computations. This separation ensures that FHE operations do not block normal contract execution and enables horizontal scaling of the computation layer independently of the consensus layer.

B. System Components

On-Chain Layer (Fhenix L2): Built on the Arbitrum Nitro stack, the on-chain layer manages encrypted state using the `FHE.sol` library. It handles vendor registration, buyer payments in USDC, token commitment storage via keccak256, rate limit enforcement at issuance time, and the vendor marketplace registry.

CoFHE Coprocessor: A specialized, stateless engine that offloads heavy FHE tasks from the main execution thread. Published benchmarks report significant throughput improvements for simple FHE operations (addition, comparison). Complex operations such as homomorphic MAC computation exhibit substantially higher latency and represent the primary technical risk for the target architecture.

Threshold Services Network (TSN): A decentralized network of nodes employing Threshold FHE Decryption and MPC Rounding [10] to securely re-encrypt or reveal data. The TSN operates on an $N/2$-of-N trust model: no single node can decrypt unilaterally, preventing insider attacks. Secured via EigenLayer restaking, providing economic accountability for threshold operations.

DePIN Proxy Network: A permissionless network of independently operated validator nodes positioned between API buyers and vendor endpoints. Nodes stake USDC collateral, validate derived keys locally via HMAC recomputation without blockchain calls, forward valid requests to vendor APIs, and earn per-request fees. This layer eliminates the vendor proxy as a single point of failure.

Vendor Marketplace: An on-chain registry where vendors list APIs, pricing tiers, and access rules. Non-custodial: funds flow directly from buyer to smart contract with automatic distribution to vendors and nodes.

C. Comparison with Existing Solutions

Table I compares KeyForge against traditional API keys and OAuth 2.0 across the dimensions most relevant to security and decentralization.

TABLE I
COMPARISON: TRADITIONAL API KEYS VS. OAUTH 2.0 VS. KEYFORGE

Property	API Keys	OAuth 2.0	KeyForge
Rules enforcement	Vendor DB	Auth server	Cryptography
Secret location	Vendor server	Auth server	FHE on-chain
Key binding	None (bearer)	User session	Wallet identity
Validation	Network call	Network call	Local HMAC
SPOF	Vendor server	Auth server	None (DePIN)
Auditable	No	No	On-chain
Post-quantum	No	No	Yes (K_m only)

IV. Cryptographic Stack

A. secp256k1 (ECDSA)

Used for vendor registration and buyer wallet binding. The vendor's Ethereum wallet signs the registration transaction.

The buyer's wallet address is embedded as an explicit input to the HMAC derivation function, binding every derived key to a specific on-chain identity. Standard Ethereum tooling; no custom implementation required.

B. HMAC-SHA256 — Key Derivation

The key derivation primitive for the PoC implementation. Given master key K_m and a structured payload, the derived key is computed as:

$$dk = \text{HMAC-SHA256}(K_m,\ W \| N \| M \| E) \tag{1}$$

where W is the buyer's wallet address, N is a cryptographically random nonce, M is the maximum use count, and E is the expiry timestamp. The derived key is deterministic: given identical inputs, the same dk is produced. Inversion is computationally infeasible: possession of dk reveals nothing about K_m. Validation at a DePIN node requires only the locally cached K_m; no network call is required. Validation latency is sub-millisecond.

C. Keccak-256 — On-Chain Commitment

The derived key itself never appears on-chain. Instead, its hash is recorded:

$$c = \text{keccak256}(dk) \tag{2}$$

Commitment c proves that key dk was legitimately issued without revealing dk. It also anchors the on-chain use counter: the contract increments a counter keyed by c upon each validated usage report from DePIN nodes.

D. TFHE / BFV — Master Key Encryption

TFHE [6] operates on binary circuits optimized for gate-by-gate operations. BFV [9] operates on integer arithmetic more efficiently for polynomial computations. The master key K_m is stored as `euint256`, a 256-bit encrypted integer equivalent in security to AES-256. The FHE layer is post-quantum secure via lattice-based cryptography.

The ECDSA layer (wallet signatures, EigenLayer) is *not* post-quantum secure. The post-quantum property applies specifically to the confidentiality of encrypted on-chain state.

E. Homomorphic MAC (Open Research)

The MAC primitive for on-chain token generation must be TFHE-compatible. Standard constructions are incompatible: Poseidon hash is designed for ZK-SNARK prime-field arithmetic and maps poorly to TFHE's binary circuits; its round function involves field exponentiations that are prohibitively expensive in binary gate representation. HMAC-SHA256 similarly requires bitwise operations at a scale that exceeds current CoFHE throughput for interactive workloads. Candidate FHE-native alternatives include LowMC [11], Kreyvium [12], and custom constructions based on TFHE-native integer addition, multiplication, and bitwise operations. The selection of a suitable MAC primitive is an **open research question** (see Section IX).

F. Symmetric Wrapping (Open Research)

Converting FHE-encrypted tokens into standard symmetric ciphertext for delivery requires trans-ciphering: computing AES decryption inside the FHE circuit. AES trans-ciphering under TFHE is prohibitively expensive at current performance levels (minutes per token). FHE-friendly alternatives include Pasta [13] and Elisabeth [14], designed for efficient homomorphic evaluation. Practical latency benchmarks for these candidates on CoFHE hardware are required (see Section IX).

Table II summarizes the role and status of each primitive.

TABLE II
CRYPTOGRAPHIC PRIMITIVE RESPONSIBILITIES

Primitive	Role	Status
secp256k1	Wallet binding	Production ready
HMAC-SHA256	Key derivation, validation	Production ready
Keccak-256	On-chain commitment	Production ready
TFHE/euint256	K_m storage & ACL	Production ready
Homomorphic MAC	On-chain token gen.	Open research
Symmetric wrap	Token delivery	Open research

V. PROTOCOL DESIGN: THE BLIND COURIER PROTOCOL

A. The Blind Courier Abstraction

The system treats the AI agent as a Blind Courier. The agent receives an Access Token encrypted specifically for the Service Vendor. The agent can forward this token to prove permission to use a service but cannot decrypt it to recover the underlying secret. This property is enforced by cryptography, not policy: the token is encrypted under the vendor's public key via the TSN. No amount of memory inspection or model introspection allows the agent to recover K_m.

B. Phase I: Provisioning

The vendor generates K_m locally on a secured system. K_m is encrypted client-side using the Fhenix FHE public key, producing a ciphertext $\tilde{K}_m$. The vendor submits $\tilde{K}_m$ to the smart contract along with SLA parameters: pricing tiers, TTL ranges, rate limits, and compliance gate specifications. After this step, the vendor's server is not required for token issuance in the target architecture.

C. Phase II: Token Generation

The buyer (or AI agent on behalf of a user) submits payment in USDC to the smart contract. The contract verifies payment, evaluates compliance rules against encrypted identity attestations, and invokes the CoFHE coprocessor to compute:

$$T_{enc} = \text{TFHE-MAC}(K_m,\ W \| N \| M \| E) \tag{3}$$

where T_{enc} is an encrypted token ciphertext. In the PoC, this step is performed off-chain by the vendor server following detection of a `KeyIssued` event.

D. Phase III: Re-encryption and Delivery

The encrypted token T_{enc} is submitted to the TSN for re-encryption under the buyer's public key PK_A. The TSN, operating under $N/2$-of-N threshold consensus, produces $T_{wrap} = \text{AES-GCM}_{PK_A}(T_{enc})$. The buyer receives T_{wrap}; decryption with their private key recovers the Access Key. No TSN node sees the plaintext token at any point.

E. Phase IV: DePIN Execution

The buyer includes the Access Key in the HTTP `Authorization: Bearer` header of every API request. DePIN nodes receive the request and perform local HMAC validation:

1) Recompute $dk' = \text{HMAC-SHA256}(K_m, W \| N \| M \| E)$ using cached K_m
2) Verify $dk' = dk$ (token integrity)
3) Verify $\text{timestamp} < E$ (not expired)
4) Verify $N \notin \mathcal{S}_{seen}$ (replay prevention)
5) If M-bounded: verify on-chain counter $< M$

Valid requests are forwarded to the vendor API endpoint. Responses are returned to the buyer. The node submits a signed usage report to the chain and claims its fee allocation.

F. Nonce Management

Nonces are random (not sequential) to avoid the serialization bottleneck of maintaining a monotonic encrypted counter under FHE. The vendor maintains a time-bucketed bloom filter for seen-nonce tracking. Two bloom filters represent the current and previous TTL window. On each request, both are checked. At each TTL boundary, the oldest is discarded and a fresh filter is initialized. Memory usage is bounded at $O(\lambda \cdot r_{TTL})$ where λ is the target false-positive rate and r_{TTL} is the expected request count per TTL window.

G. Rate Limit Enforcement

Rate limits are enforced through a hybrid model:

At issuance (on-chain): The contract tracks total tokens issued per subscription period. Over-issuance is prevented at the protocol level.

At validation (off-chain): DePIN nodes track per-token usage locally. Usage reports are aggregated in Merkle rollups and submitted in batches to minimize gas costs.

Rate limits are *advisory* at the per-request level and *enforced* at the issuance level. Per-request on-chain enforcement is infeasible; issuance-time gating bounds total access.

H. Worked Example: 10-Use Token Lifecycle

Consider a buyer purchasing a 10-use token with nonce $N = \texttt{0xA3F2}$ and expiry $E = t + 3600$. Table III traces the state at key points.

On the 11th call, the counter check fails and the request is rejected with HTTP 429 before it reaches the vendor endpoint. This rejection is provable on-chain via the committed use counter.

TABLE III
TOKEN LIFECYCLE: 10-USE TOKEN

Call	Nonce check	Counter	Result
1	$N \notin \mathcal{S}$	1 / 10	FORWARD
5	$N \notin \mathcal{S}$	5 / 10	FORWARD
10	$N \notin \mathcal{S}$	10 / 10	FORWARD
11	—	10 / 10	REJECT (limit)
—	$N \in \mathcal{S}$	—	REJECT (replay)

VI. DEPIN ECONOMICS

A. Node Registration and Staking

Any party may join the validator network by submitting the minimum stake S_{min} in USDC and deploying the open-source node software. Stake is locked in the smart contract and subject to slashing.

```
function registerNode(uint256 stakeAmount) external {
    require(stakeAmount >= MIN_STAKE,
            "insufficient stake");
    usdc.transferFrom(msg.sender,
                      address(this), stakeAmount);
    nodeStakes[msg.sender] = stakeAmount;
    emit NodeRegistered(msg.sender, stakeAmount);
}

function slashNode(address node,
    bytes calldata proof) external onlyGovernance {
    uint256 stake = nodeStakes[node];
    nodeStakes[node] = 0;
    emit NodeSlashed(node, stake);
}
```

Listing 1. Node Registration and Slashing

B. Fee Distribution

Every processed request triggers a micro-payment from the pre-paid buyer allocation. For a request priced at P USDC, the distribution is:

TABLE IV
FEE DISTRIBUTION PER REQUEST

Recipient	Share	Rationale
Service Vendor	80%	API service provision
DePIN Node	15%	Validation + forwarding
Protocol Treasury	5%	Development, governance

C. Node Economics: Worked Example

Consider a network handling 1,000,000 requests per day at \$0.001 per request, distributed across 100 nodes of equal throughput:

$$\text{Daily network revenue} = 1{,}000{,}000 \times \$0.001 = \$1{,}000 \tag{4}$$

$$\text{Node share (15\%)} = \$150 \text{ per day (network total)} \tag{5}$$

$$\text{Per-node daily earnings} = \$150/100 = \$1.50 \tag{6}$$

$$\text{Annualized per node} \approx \$547.50 \tag{7}$$

At a minimum stake of $500 USDC, annual yield is approximately 109%. As network usage scales, per-node earnings scale linearly assuming constant node count, creating natural incentive for network growth.

D. Slashing Conditions

Nodes face slashing for provable misbehavior:

- **Dropped requests:** Buyer receives no response but node claims fee. Detectable via signed request receipts and buyer complaint.
- **Invalid validation:** Node forwards a request with an invalid derived key. Detectable via on-chain commitment mismatch.
- **TSN collusion:** Node participates in unauthorized decryption. Detectable via threshold cryptography proofs.

VII. Security Analysis

A. Threats Addressed

Credential theft at rest: FHE encryption ensures K_m has no plaintext representation on-chain. A complete blockchain state dump reveals only the ciphertext $\tilde{K}_m$, which is computationally indistinguishable from random under the TFHE security assumption.

Credential theft in transit: Key delivery via TSN re-encryption wraps the Access Token specifically for the buyer's public key. Only the buyer's private key can unwrap it. No TSN node observes the plaintext token.

Replay attacks: Random nonces embedded in every token, combined with vendor-side bloom filter tracking, prevent replaying captured valid requests.

Over-issuance: On-chain rate limits at issuance time prevent any party from issuing tokens beyond the subscription quota, even in the event of a compromised vendor server.

Proxy node collusion: EigenLayer staking combined with $N/2$-of-N threshold trust for the TSN requires a majority of nodes to collude simultaneously to break security guarantees. This is economically irrational given slashing.

Bearer token sharing: Wallet-bound key derivation (Equation 1 embeds W as an HMAC input) means a derived key for wallet W_A is invalid when presented from wallet W_B. The keys cannot be usefully transferred.

B. Threats Not Addressed

Vendor machine compromise: If an attacker compromises the vendor's local system holding K_m, all derived keys can be forged. FHE protects K_m from all parties *except* the vendor. This is a deliberate design choice: the vendor must trust their own infrastructure for key custody.

Majority TSN collusion: Compromise of more than $N/2$ TSN nodes simultaneously allows unauthorized decryption. EigenLayer staking makes this economically costly but not cryptographically impossible.

Quantum attacks on ECDSA: The Ethereum wallet layer (secp256k1 signatures, EigenLayer restaking) is not post-quantum secure. A sufficiently capable quantum adversary could forge wallet signatures. The FHE layer's lattice-based security is unaffected.

C. Trust Assumptions (Enumerated)

1) Vendor's local key custody system is not compromised.
2) Fewer than $N/2$ TSN nodes collude at any time.
3) The Fhenix L2 consensus mechanism is not captured.
4) EigenLayer staking provides sufficient economic deterrence for node misbehavior.
5) TFHE security reduces to the hardness of RLWE (Ring Learning With Errors).

VIII. The AI Agent Use Case

A. The Goldfish Bowl Problem

Traditional blockchains are fully transparent. Every state variable is public. Storing API master keys, private SLA terms, or sensitive business logic on-chain exposes them to every blockchain observer, competitor, and attacker simultaneously. Off-chain key management services address transparency but reintroduce centralized trust. The "Goldfish Bowl" problem is the impossibility of maintaining secret state in a trustless transparent environment without cryptographic tools. FHE solves this: $\tilde{K}_m$ is public; K_m is not.

B. Blind Credential Handling

An AI agent acting as a Blind Courier can call a paid API without its operator ever knowing which credential was used. The agent receives T_{wrap}, forwards it in the `Authorization` header, and receives the API response. It cannot inspect the token contents. It cannot extract K_m. It cannot forge additional tokens.

This enables organizational delegation at scale: a company can grant an agent framework access to 50 different vendor APIs, with each credential independently gated by on-chain SLA rules, without exposing any credential to the agent developer.

C. Compliance Gating

The system supports compound identity checks on encrypted inputs before token issuance. A vendor may specify: "issue tokens only to wallets with a Self Protocol KYC proof, from non-sanctioned jurisdictions, with an on-chain reputation score above threshold." All checks execute on encrypted identity attestations using FHE comparisons. The rule produces a boolean result without any plaintext identity data touching the chain or being visible to the vendor.

D. One-Step Payment vs. x402

Table V compares KeyForge session tokens against the x402 per-request payment protocol for a workload of 1,000 API calls.

IX. Open Research Problems

A. FHE-Friendly MAC Primitive Selection

The problem: On-chain token generation (Equation 3) requires a MAC computable inside TFHE's binary circuit model. Standard MACs are incompatible: HMAC-SHA256 requires bit-level operations at a scale that exceeds practical

TABLE V
PAYMENT MODEL COMPARISON: 1,000 API CALLS

Property	x402	KeyForge
Round-trips per call	3–4	1
On-chain txns	1,000	1 (issuance)
Gas cost	$1000 \times g$	g
Payment latency	Per-request	Pre-paid
Credential exposure	Per-request	Never
Offline validation	No	Yes

FHE circuit depth; Poseidon hash uses field exponentiations that map poorly to binary gates.

Why it matters: Solving this enables the full target architecture where the vendor is completely offline after provisioning. Token issuance becomes trustless, non-interactive, and 24/7 without vendor availability.

Candidates:

- **LowMC** [11]: Designed to minimize AND-gate depth in circuits, making it FHE-friendly. Security proofs exist; no production TFHE implementation is publicly available as of this writing.
- **Kreyvium** [12]: A stream cipher variant of Trivium designed for FHE evaluation. Lower circuit depth than AES; full MAC construction requires additional design.
- **Custom TFHE constructions**: Integer addition and multiplication chains native to TFHE that approximate MAC security properties. Cryptographic soundness proofs required.

Research timeline: An active research area; production-ready candidates are estimated 12–24 months from independent benchmarking to production deployment.

B. Symmetric Wrapping Performance

The problem: Converting FHE-encrypted tokens into AES-GCM ciphertext for delivery (Phase III) requires evaluating AES decryption inside the FHE circuit. AES trans-ciphering under TFHE currently takes minutes per token, making interactive token delivery impractical.

Why it matters: Solving this enables the TSN to deliver tokens in real-time without the vendor server participating in wrapping. Combined with the MAC primitive, it completes the vendor-offline guarantee.

Candidates:

- **Pasta** [13]: A family of FHE-friendly ciphers designed to minimize multiplicative complexity. Preliminary benchmarks are promising.
- **Elisabeth** [14]: A filter permutator construction specifically optimized for TFHE evaluation. Smaller circuit than AES.

Research timeline: 6–18 months to practical deployment given current trajectory of FHE performance improvements.

C. Resale and Homomorphic Rate-Limit Subdivision

The problem: A resource owner with 50% of their monthly quota remaining should be able to sell that remainder to a third party without exposing the underlying credential. This requires homomorphic subdivision of an encrypted use counter: splitting a token granting M uses into two tokens granting M_1 and $M_2 = M - M_1$ uses respectively, with cryptographic double-spend prevention.

Why it matters: Unlocks a secondary market for API access, enabling monetization of underutilized enterprise subscriptions without credential exposure.

Research status: No complete cryptographic construction exists. The problem requires defining homomorphic operations on encrypted counters combined with a nullifier scheme to prevent double-spending. Deferred to a future specification version.

X. IMPLEMENTATION ROADMAP

A. Phase 1: PoC (Buildable Today)

The following components are fully implementable with current tooling:

- `euint256` master key storage on Fhenix devnet with ACL-gated vendor-only decryption
- Off-chain HMAC-SHA256 key derivation on vendor server
- On-chain keccak256 commitment storage and use counter
- DePIN node prototype: local HMAC validation, request forwarding
- USDC payment integration via Circle
- CLI demo: register $\rightarrow$ buy $\rightarrow$ call $\times 10 \rightarrow$ reject on call 11

Demo target: Three terminal windows demonstrating the full PoC flow. The 11th-call rejection is the proof moment: visible, unambiguous, and cryptographically verifiable via the on-chain counter.

B. Phase 2: TSN Integration

- Integrate with production TSN (Lit Protocol or custom threshold network)
- Implement re-encryption delivery for buyer public keys
- Vendor becomes fully offline for token issuance
- Blocked on: MAC primitive selection (Section IX)

C. Phase 3: Full DePIN Economics

- Production DePIN node network with EigenLayer staking
- Batched Merkle usage reporting
- On-chain slashing with fraud proof verification
- Resale market (blocked on homomorphic subdivision research)
- Multi-chain deployment via LayerZero CCIP

XI. ON-CHAIN IMPLEMENTATION

The core Solidity contracts implement encrypted master key storage with authorized computation, DePIN node management, and token commitment tracking.

```
import "fhevm/lib/TFHE.sol";

contract KeyForgeRegistry {
    // Encrypted 256-bit master key per vendor
    mapping(address => euint256) private masterKeys;
```

```
    // Commitment -> use counter
    mapping(bytes32 => uint256) public usageCounts;

    // Token validity (revocation)
    mapping(uint256 => bool) public tokenStatus;

    // DePIN node stakes
    mapping(address => uint256) public nodeStakes;

    function storeMasterKey(
        einput calldata encryptedKey,
        bytes calldata proof
    ) external {
        euint256 km = TFHE.asEuint256(
            encryptedKey, proof);
        masterKeys[msg.sender] = km;
        // Vendor-only ACL: only msg.sender can decrypt
        TFHE.allow(km, address(this));
        TFHE.allow(km, msg.sender);
        emit MasterKeyStored(msg.sender);
    }

    function recordCommitment(
        bytes32 commitment,
        uint256 maxUses
    ) external {
        usageCounts[commitment] = 0;
        // maxUses stored as SLA parameter
        emit KeyIssued(msg.sender, commitment, maxUses);
    }

    function reportUsage(
        bytes32 commitment
    ) external onlyNode {
        usageCounts[commitment] += 1;
        emit UsageReported(commitment,
            usageCounts[commitment]);
    }
}
```

Listing 2. Master Key Storage with FHE ACL

```
function deriveKey(Km, wallet, nonce,
                   max_uses, expiry):
    payload = wallet || nonce
            || uint64(max_uses)
            || uint64(expiry)
    dk = HMAC-SHA256(Km, payload)
    commitment = keccak256(dk)
    return dk, commitment

function validateKey(dk, Km, wallet, nonce,
                     max_uses, expiry):
    expected = deriveKey(Km, wallet, nonce,
                         max_uses, expiry)
    if dk != expected.dk: return INVALID
    if now() > expiry:    return EXPIRED
    if nonce in seen_set: return REPLAY
    return VALID
```

Listing 3. HMAC Key Derivation (Pseudocode)

XII. Conclusion

KeyForge introduces a cryptographically grounded approach to API access control that addresses three fundamental failures of the current model: credential durability, policy-based enforcement, and agent credential exposure. The three core contributions — FHE-protected master key storage, HMAC-SHA256 wallet-bound key derivation, and DePIN proxy validation — are independently meaningful and composable.

The PoC implementation is fully buildable with current tooling on Fhenix L2. The full target architecture, in which vendors require no online infrastructure after initial provisioning, awaits resolution of the FHE-friendly MAC primitive selection problem. This represents a genuine research gap, not an engineering challenge, and is clearly characterized as such.

The DePIN economic model draws on validated precedents (Helium, Filecoin, Render) and provides quantifiable incentives for node operators at network launch. The AI agent use case represents an immediately applicable deployment target: organizations running autonomous agents on third-party APIs today have no cryptographic mechanism to prevent credential exposure. KeyForge provides one.

Researchers are invited to contribute to the three open problems identified in Section IX. Resolution of the MAC primitive selection problem in particular would unlock the full vendor-offline architecture and represent a meaningful advance in practical FHE deployment.

References

[1] GitHub Security Lab, "The state of secret sprawl," GitHub Inc., 2024. Available: https://github.blog/security/

[2] Kong Inc., "Kong Gateway Documentation," 2024. Available: https://docs.konghq.com

[3] Amazon Web Services, "Amazon API Gateway Developer Guide," AWS, 2024. Available: https://docs.aws.amazon.com/apigateway

[4] Lit Protocol Foundation, "Lit Protocol: Decentralized Access Control," 2023. Available: https://litprotocol.com

[5] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[6] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachene, "TFHE: Fast Bootstrapping over the Torus," *Journal of Cryptology*, vol. 33, pp. 7–37, 2020.

[7] Fhenix Foundation, "Fhenix: Confidential Smart Contracts using FHE," 2024. Available: https://fhenix.io

[8] Fhenix Research, "CoFHE: Collaborative FHE Coprocessor Architecture," 2024.

[9] J. Fan and F. Vercauteren, "Somewhat Practical Fully Homomorphic Encryption," *IACR ePrint*, 2012/144, 2012.

[10] E. Ben-Sasson et al., "MPC Rounding Protocol for Threshold Decryption," *Proceedings of CCS 2025*, 2025.

[11] M. Albrecht et al., "Ciphers for MPC and FHE," *Advances in Cryptology – EUROCRYPT 2015*, pp. 430–454, 2015.

[12] A. Canteaut et al., "Stream Ciphers: A Practical Solution for Efficient Homomorphic-Ciphertext Compression," *Fast Software Encryption 2016*, LNCS vol. 9783, pp. 313–333, 2016.

[13] L. Grassi et al., "Pasta: A Case for Hybrid Homomorphic Encryption," *IACR Transactions on Symmetric Cryptology*, 2021.

[14] C. Cosseron et al., "Towards Case-Optimized Hybrid Homomorphic Encryption," *ASIACRYPT 2022*, 2022.

[15] Coinbase Developers, "x402: HTTP 402 Payment Protocol," 2024. Available: https://x402.org

[16] Helium Foundation, "Helium Network Whitepaper," 2023. Available: https://helium.com

[17] Protocol Labs, "Filecoin: A Decentralized Storage Network," *Protocol Labs Whitepaper*, 2017.

[18] Render Foundation, "Render Network: Decentralized GPU Compute," 2023. Available: https://rendernetwork.com